Studio Nx

JS 108 — Transport

2026-10-14

JS 108 — `Transport` (재생 스케줄러 루프)

항목	내용
파일	`studionx/frontend/static/js/audio/scheduler.js` (같은 파일에 클래스 추가)
작업	S1-05 오디오 엔진 기초: 다섯 번째 함수 (FR-201 재생/정지, FR-205 tick/PPQ 960)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-05-audio-engine`
다음	pause/resume와 재생 중 위치 이동(seek), 재생 중 템포 변경, 그다음 메트로놈, 루프

1. 하는 일

107의 `planSchedule`(순수 계산)을 실제 소리 시각에 얹는 루프입니다. `clock`(브라우저에서는 `AudioContext`)에서 지금 시각을 읽어 "앞으로 lookahead(0.1초) 안에 울릴 소리"를 `onStep(tick, time)`으로 넘기고, 커서를 옮깁니다. 타이머(약 25ms)가 이 `pump()`를 반복해서 부릅니다.

값	기본	뜻
`lookahead`	0.1초	앞으로 이만큼 미리 예약합니다
`intervalMs`	25	타이머가 도는 간격입니다
`startDelay`	0.05초	시작할 때 첫 소리를 조금 뒤로 미룹니다
`stepTicks`	240 (1/16)	한 단계에 나아갈 tick 입니다 (PPQ/4)

`clock`(필수)과 `setTimer·clearTimer`(기본 전역 `setInterval·clearInterval`)를 주입받게 만들어, Node에서 가짜 시계와 가짜 타이머로 소리 없이 시험합니다.

2. 코드

```
export class Transport {
  constructor(options) {
    options = options || {};
    if (options.clock == null || typeof options.clock.now !== 'function') {
      throw new TypeError('Transport 에는 now() 를 가진 clock 이 필요합니다 (브라우저에서는 AudioContex
t)');
    }
    this.clock = options.clock;
    this.onStep = typeof options.onStep === 'function' ? options.onStep : function () {};
    this.baseBpm = options.baseBpm != null ? options.baseBpm : 120;
    this.changes = options.changes != null ? options.changes : [];
    this.stepTicks = options.stepTicks != null ? options.stepTicks : PPQ / 4; // 기본 1/16
    this.lookahead = options.lookahead != null ? options.lookahead : DEFAULT_LOOKAHEAD_SECONDS;
    this.intervalMs = options.intervalMs != null ? options.intervalMs : DEFAULT_INTERVAL_MS;
    this.startDelay = options.startDelay != null ? options.startDelay : DEFAULT_START_DELAY_SECONDS;
    this.setTimer = options.setTimer != null ? options.setTimer : function (fn, ms) { return setInterval
(fn, ms); };
    this.clearTimer = options.clearTimer != null ? options.clearTimer : function (id) { clearInterval(i
d); };
```

```
    this.playing = false;
    this.timerId = null;
    this.cursorTick = 0;
    this.cursorTime = 0;
    this.originTick = 0;
    this.originTime = 0;
  }

  start(fromTick) {
    this.stop();
    this.playing = true;
    this.cursorTick = fromTick != null ? fromTick : 0;
    this.originTick = this.cursorTick;
    this.cursorTime = this.clock.now() + this.startDelay;
    this.originTime = this.cursorTime;
    this.pump();
    const self = this;
    this.timerId = this.setTimer(function () { self.pump(); }, this.intervalMs);
    return this;
  }

  stop() {
    if (this.timerId != null) {
      this.clearTimer(this.timerId);
      this.timerId = null;
    }
    this.playing = false;
    return this;
  }

  pump() {
    const untilTime = this.clock.now() + this.lookahead;
    const plan = planSchedule(this.cursorTick, this.cursorTime, untilTime, this.stepTicks, this.baseBpm,
this.changes);
    for (let i = 0; i < plan.events.length; i++) {
      this.onStep(plan.events[i].tick, plan.events[i].time);
    }
    this.cursorTick = plan.nextTick;
    this.cursorTime = plan.nextTime;
    return plan;
  }

  positionTick() {
    const originSeconds = ticksToSecondsWithChanges(this.originTick, this.baseBpm, this.changes);
    const nowSeconds = originSeconds + (this.clock.now() - this.originTime);
    return secondsToTicks(nowSeconds, this.baseBpm, this.changes);
  }
}
```

최신 문법으로 쓰면 (공부용)

```
const self = this;
this.timerId = this.setTimer(function () { self.pump(); }, this.intervalMs);
// 위 두 줄은 화살표 함수로:
this.timerId = this.setTimer(() => this.pump(), this.intervalMs);
```

최신 문법	실제 코드	뜻
화살표 함수 `() => this.pump()`	`const self = this; function () { self.pump(); }`	화살표 함수는 `this` 가 만들어질 때의 것을 그대로 씁니다. 실제 코드는 `self` 에 담아 씁니다
`options.clock ?? ...`	`options.clock != null ? … : …`	`null`·`undefined` 일 때 기본값을 씁니다
`class`	(같음)	실제 코드도 `class` 를 씁니다

3. 왜 이렇게

선택	이유
`clock` 을 주입받음	브라우저에서는 `AudioContext`(그 `currentTime`)를 그대로 넘기고, 시험에서는 가짜 시계를 넘겨 소리 없이 확인합니다. 나중에 오프라인 렌더(`OfflineAudioContext`, S1-11)도 같은 방식으로 씁니다
`setTimer`·`clearTimer` 도 주입	Node 시험에서 진짜 타이머를 기다리지 않고 손으로 `pump()` 를 불러 1초를 한 번에 시뮬레이션합니다
기본 타이머를 함수로 감쌈	`this.setTimer(...)` 로 전역 `setInterval` 을 그대로 부르면 `this` 가 `window` 가 아니게 되어 브라우저에서 "Illegal invocation" 이 납니다. `function (fn, ms) { return setInterval(fn, ms); }` 로 감싸 `this` 와 무관하게 부릅니다
`start` 에서 곧바로 한 번 `pump()`	첫 소리도 lookahead 안에 미리 걸려 늦지 않습니다. 타이머 첫 회(25ms 뒤)만 기다리면 시작 직후가 빕니다
`startDelay`	커서를 `now` 가 아니라 `now + 0.05` 에서 시작해, 이미 지난 시각에 예약하는 일을 막습니다
`start` 가 먼저 `stop()`	두 번 눌러도 타이머가 겹치지 않습니다
`positionTick` 은 기준점 계산	커서가 아니라 "재생 시작점(originTick, originTime)"에서 흐른 시간을 105·106으로 되돌립니다. 예약을 미리 해 둬도 화면 재생선은 실제 시각과 맞습니다
이번엔 재생·정지·위치만	pause/resume, seek, 재생 중 템포 변경은 예약 취소·재계산이 필요해 다음 함수로 미룹니다. `stop`은 타이머만 끊으므로 이미 예약된 창(최대 0.1초)의 소리는 울린 뒤 멈춥니다

4. 시험 (Node에서 직접 실행, 23개 통과 · 가짜 시계·가짜 타이머)

확인	결과
기본값(lookahead 0.1, interval 25ms, startDelay 0.05)	✓
`clock` 없으면 `TypeError`	✓
`start` 하면 곧바로 한 번 예약, 첫 소리가 startDelay(0.05초)에, 타이머 등록	✓
25ms 씩 1초 시뮬레이션: tick이 240씩 이어지고 소리 간격이 0.125초, 늦게 예약한 소리 없음	✓
`stop` 하면 타이머 제거, 더 이상 예약 안 함	✓
`start(3840)` 위치 기준으로 시작	✓
`positionTick`: 시작 시점·+0.125초(240tick)·+0.5초(960tick)	✓
템포 변경 곡에서 `positionTick`(60 BPM 0.5초 = 480tick)	✓
`start` 두 번 = 앞 타이머 정리(겹치지 않음)	✓